

Week 1 - Day 4a: Loop Unrolling and Debugging

In this exercise, we will practice several important programming skills that are essential for writing and debugging code. You will practice:

1. **Loop Unrolling:**
 - Writing out each iteration of a loop explicitly.
2. **Conditionals and Loop Interaction:**
 - Understanding how if statements and loops work together.
3. **Program Tracing:**
 - Understanding the step-by-step execution of code.
 - Keeping track of variable values throughout the program.
4. **Identifying and Fixing Mistakes:**
 - Tracing the program to identify logical errors.
 - Correcting the code to achieve the desired output.
5. **Common Programming Errors:**
 - Logical errors in loops and conditionals.
 - Misplacement of code blocks within loops and conditionals.

Exercise 1: Unrolling for Loops []

Loops are used to **repeat** the content of their **body** multiple times. To understand what exactly is happening inside a loop, we can **unroll** the loop, which means writing out each iteration of the loop explicitly. Let's look at an example:

Example 1.1 Consider the following loop:

```
for i in [1, 4, 3, 6]:  
    print(i) # loop body
```

To unroll, we repeat the body for each repetition to get the exact same result:

```
# First repetition  
i = 1 # assignment of loop variable  
print(i) # loop body  
  
# Second repetition  
i = 4 # assignment of loop variable  
print(i) # loop body  
  
# Third repetition  
i = 3 # assignment of loop variable  
print(i) # loop body  
  
# Fourth repetition
```

```
i = 6      # assignment of loop variable
print(i)   # loop body
```

If you run each of the code pieces, both produce the **same result**. This is because the unrolled loop explicitly writes out each step that the original loop would perform.

Sometimes, unrolled loops can become very long, which is why we use `for` and `while` loops in the first place! Let's look at another example where the unrolled loop is a little more complex.

Example 1.2 Here's a loop that calculates the sum of odd numbers from 0 to 5:

```
# Initialize the total sum to 0
total = 0

# Loop through numbers from 0 to 5 (inclusive)
for i in range(6): # range(6) generates numbers [0, 1, 2, 3, 4, 5]
    if i % 2 == 1: # Check if the number is odd (remainder when
        # divided by 2 is 1)
        print(i)
        total += i

# Print the total sum of odd numbers
print("Total sum of odd numbers up to 5:", total)
```

Again, we unroll this loop by writing each repetition of the loop explicitly.

```
# Sum up odd numbers up to 5
total = 0

# First repetition
i = 0 # assignment of loop variable
if i % 2 == 1:
    print(i)
    total += i

# Second repetition
i = 1 # assignment of loop variable
if i % 2 == 1:
    print(i)
    total += i

# Third repetition
i = 2 # assignment of loop variable
if i % 2 == 1:
    print(i)
    total += i

# Fourth repetition
i = 3 # assignment of loop variable
if i % 2 == 1:
```

```

    print(i)
    total += i

# Fifth repetition
i = 4 # assignment of loop variable
if i % 2 == 1:
    print(i)
    total += i

# Sixth repetition
i = 5 # assignment of loop variable
if i % 2 == 1:
    print(i)
    total += i

# LOOP FINISHED
# Code that follows the loop
print("Total sum of odd numbers up to 5:", total)

```

Again, If you run both the original loop and the unrolled loop, they will produce the **same result**. This demonstrates that unrolling a loop doesn't change the logic or outcome of the program; it just makes each step explicit.

Let's practice unrolling loops with some exercises. Recall that unrolling a loop means writing out each repetition explicitly.

1.1 Consider the following code:

```

sum_of_squares = 0

for y in [1, 2, 3, 4]:
    sum_of_squares += y * y # loop body

sum_of_squares

```

Unroll the loop by writing out each repetition of the loop explicitly!

```

# TODO: Unroll the loop

```

Run your code and make sure it shows the same result as the original code!

1.2 Look at the following code from the lecture.

```

x = [1, 2, 3, 4]
y = [5, 6, 7, 8]
mul_sum = 0

for ind in range(len(x)):
    product = x[ind] * y[ind]

```

```
    mul_sum += product
mul_sum
```

1.2.1 What is `len(x)`?

```
# TODO: Put your answer here
```

1.2.2 Expressed as a list, what is `range(len(x))`?

```
# TODO: Put your answer here
```

1.2.3: Now **unroll** the loop by writing out each repetition of the loop explicitly!

```
# TODO: Unroll the code
```

Run your code and make sure it shows the same result as the original code!

Exercise 2: Unrolling While Loops []

You can also unroll while loops. This is a bit trickier because you don't know in advance how many times the loop will run. Let's look at an example.

Example 2.1 Consider the following code that prints the powers of 2 less than 25.

```
n = 1
while n < 25:
    print(n)      # Print the current power of 2
    n *= 2        # Multiply n by 2 to get the next power of 2
```

To unroll this loop, we need to keep track of the values of the variables, so that we know when the **condition** becomes false and the loop finishes. The unrolled loop is shown below:

```
# Print powers of 2 less than 25
n = 1

# 1
print(n)  # Print the current value of n (which is 1)
n *= 2    # Double n, now n = 2

# 2
print(n)  # Print the current value of n (which is 2)
n *= 2    # Double n, now n = 4

# 3
print(n)  # Print the current value of n (which is 4)
n *= 2    # Double n, now n = 8
```

```
# 4
print(n) # Print the current value of n (which is 8)
n *= 2 # Double n, now n = 16

# 5
print(n) # Print the current value of n (which is 16)
n *= 2 # Double n, now n = 32

# Now n = 32, which is not less than 25, so we stop
```

If you run each of these code pieces, they will produce the **same result**. Unrolling loops helps you see each step in the process, which can make understanding and debugging your code easier.

2.1 Look at the following code from the lecture which computes the sum of the first 5 square numbers.

```
i = 1
summ = 0
while i <= 5:
    summ += i**2
    i += 1
print(summ)
```

Unroll the loop by writing out each repetition of the loop explicitly!

```
# TODO: Unroll the loop
```

Run your code and make sure it shows the same result as the original code!

Example 2.2 Let's look at a more complex example using if statements.

```
lst = [41, 2, 89]
i = 0
while i < len(lst):
    if lst[i] > 20:
        print(lst[i])
    i += 1
print("Done.")
```

After unrolling, we get the following code:

```
lst = [41, 2, 89]
i = 0

# 1
if lst[i] > 20: # Check if the first element (41) is greater than 20
    print(lst[i]) # Print 41 because it is greater than 20
```

```

i += 1          # Increment i, now i = 1
# 2
if lst[i] > 20:  # Check if the second element (2) is greater than
20              # 20
    print(lst[i]) # 2 is not greater than 20, so nothing is printed
i += 1          # Increment i, now i = 2
# 3
if lst[i] > 20:  # Check if the third element (89) is greater than
20              # 20
    print(lst[i]) # Print 89 because it is greater than 20
i += 1          # Increment i, now i = 3
print("Done.")  # Print "Done." to indicate the loop is finished

```

If you run each of the code pieces, both produce the **same result**. Notice how we need to keep track of the value of `i` in order to know whether the while loop repeats or stops.

When analyzing and understanding code, it often helps to keep track of all variables that appear in the code. That way, we can also see what happens in the loop body, for example, if some `if` condition is true or false.

To do this systematically, we can do something called **program tracing**.

Exercise 3: Program Tracing []

Let's talk about tracing your program. Tracing involves stepping through the program line by line and figuring out what happens after each line of code. This is a very powerful way to understand what is going on in your program!

Consider this program:

```

a = 3          # Line 1
b = 5          # Line 2
temp = a       # *Line 3*
a = b          # Line 4
b = temp       # Line 5

```

3.1: What is the value of `a`, `b`, and `temp` **after** Line 3?

```

# TODO: Fill in the blanks!
a = ...
b = ...
temp = ...

```

A tracing table has one line of code on each row and all the variables in the program as columns. Next to each line of code, you write down the value of each variable **after** that line of code executes. Tracing tables are helpful to understand how variables change throughout the

program. Let's put the program above into a tracing table and see what the tracing table looks like up to Line 3.

Code	a	b	temp
a = 3 # Line 1	3		
b = 5 # Line 2	3	5	
>> temp = a # Line 3	3	5	3
a = b # Line 4			
b = temp # Line 5			

****>> indicates where the program is currently.**

3.2: Now fill in the rest of the table (Line 4 and Line 5) on a piece of paper before proceeding.

Exercise 4: Tracing Loops []

Tracing is a bit trickier with loops. You will need to repeat the table for the loop one time per pass. Sometimes you don't know when the loop will end when you start tracing. For conditionals, you will ignore rows that do not get executed. The specific format of the tracing table is not important; what is important is that you understand it yourself and that it helps you see what the program is doing.

Take a look at this program from Day 3, which we just unrolled earlier:

```
lst = [41, 2, 89]
i = 0
while i < len(lst):
    if lst[i] > 20:
        print(lst[i])
    i += 1
```

Because there is a loop, the table for this program will contain sections for multiple passes. This looks a bit like the unrolled loop in the earlier section.

Code	i	lst[i]	condition
lst = [41, 2, 89]			
i = 0	0		
Pass 1			
while i < len(lst)	0	41	Yes

Code	i	lst[i]	condition
:			
if	0	41	Yes
lst[i] >			
20:			
print(ls	0	41	(Print 41)
t[i])			
i += 1	1	2	
Pass 2			
while i	1	2	Yes
<			
len(lst)			
:			
if	1	2	Yes
lst[i] >			
20:			
print(ls			(Is skipped)
t[i])			
i += 1	2	89	
Pass 3			
while i	2	89	Yes
<			
len(lst)			
:			
if	2	89	Yes
lst[i] >			
20:			
print(ls	2	89	(Print 89)
t[i])			
i += 1	3	?	
Pass 4			
while i	3	?	No
<			
len(lst)			
:			

Notice that we added some useful columns to track like `lst[i]` and the `condition`. These are not variables per se, but they are useful to track.

Try tracing this program on a piece of paper. This is from Day 3.

4.1 Trace this program on paper

```
# I want to add (and print) the first few numbers until their sum
exceeds 100.
```

```
# but nothing is being printed
total = 0
i = 1
while total >= 100:
    total += i
    i += 1
    print(i)
```

Question 4.2: Trace this program on paper

```
# I want to print the first number in my list that's bigger than 10.
# But the answer I get, 5, is certainly not bigger than 10.
myList = [1, 3, 9, 5, -1, 13, 5, 11, 2]
number = 0
while myList[number] <= 10:
    number += 1
print(number)
```

For each of the remaining code snippets:

- Do **program tracing** to understand what is happening
- Find the mistake in the code, and put a **fixed** version in the cell below.
- Do **program tracing** on the **fixed** code.

Exercise 5 []

We want to print the odd numbers less than 6. The expected output should be 1, 3, 5.

```
num = 0
if num % 2 == 1:
    while num < 6:
        print(num)
    num += 1
```

5.1 Trace through the program on a piece of paper.

5.2 Find a mistake in the code, and put a fixed version in the cell below.

```
# TODO: Write a fixed version of the code.
```

5.3 Do program tracing on the fixed code on paper.

Exercise 6 []

I want to sum up the first 5 powers of 2 (1 + 2 + 4 + 8 + 16). The answer should be 31.

```
summ = 0
```

```
for i in range(5):
    power = 2**i
    summ += power

print(summ)
```

6.1: Trace through the program on a piece of paper.

6.2 Find a mistake in the code, and put a fixed version in the cell below.

```
# TODO: Write a fixed version of the code.
```

6.3 Do program tracing on the fixed code on paper.

Exercise 7 []

We want to put even numbers less than 5 in a list, in decreasing order. The expected output should be [4, 2, 0].

```
num = 5
myList = []
while num >= 0:
    num -= 1
    if num % 2 == 0:
        myList += [num]
myList
```

7.1: Trace through the program on a piece of paper.

7.2 Find a mistake in the code, and put a fixed version in the cell below.

```
# TODO: Write a fixed version of the code.
```

Exercise 8 []

I want to count how many numbers in my list are positive. The answer should be 3, but I get 1 instead in my code below.

```
lst = [3, -4, -7, 8, 1]

for elem in lst:
    npositive = 0
    if elem >= 0:
        npositive += 1

npositive
```

8.1: Trace through the program on a piece of paper.

8.2 Find a mistake in the code, and put a fixed version in the cell below.

```
# TODO: Write a fixed version of the code.
```

If you've completed all of the exercises so far, congratulations! There are supplementary exercises you can do now. Ask a TA to show you how to get access to them.